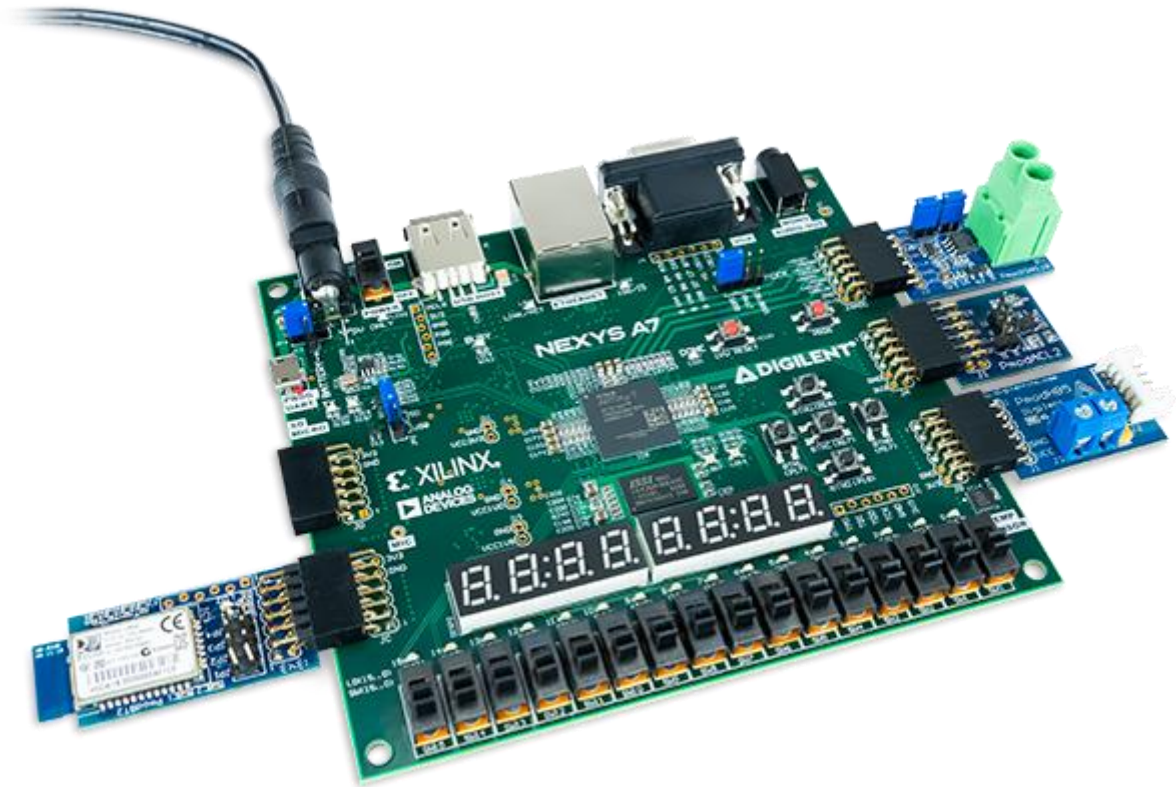


Département IEM (*Informatique Electronique Mécanique*)

M1 ESI : *Electronique, Signal et Image* - Semestre 2

Systèmes Reconfigurables (FPGA/VHDL)

Travaux Pratiques



Document préparé par **Mahdi Madani**

Contact : mahdi.madani@u-bourgogne.fr

Cours en ligne : <https://plubel-prod.u-bourgogne.fr/course/view.php?id=5629>

Sommaire

TP1 : Introduction à l'IDE Vivado	P02
TP2 : Filtre anti-rebond sur FPGA	P14
TP3 : Contrôle d'un afficheur 7 segments par un FPGA	P15
TP4 : Système séquentiel.....	P16
TP5 : Machine à états finis	P18
Annexe A : simulation par un banc de test.....	P20

TP : Introduction à l'IDE Vivado (1 séance)

Ce TP vise à découvrir le flux de conception Vivado et à mettre en œuvre une architecture numérique sur la cible FPGA Xilinx de la série Artix-7 montée sur la carte de développement Digilent Nexyx A7. Ce premier TP se déroulera sous la forme d'un tutoriel en suivant les étapes ci-dessous. Ensuite, vous concevrez un circuit combinatoire en tant qu'activité d'apprentissage. À la fin de ce tutoriel, vous serez en mesure de créer un projet Vivado et de mettre en œuvre une architecture sur une cible FPGA.

Partie 1 : création d'un projet Vivado et d'une conception VHDL

- Création d'un nouveau projet Xilinx Vivado
- Ajout de nouveaux fichiers source (écriture de code VHDL)

Partie 2 : vérification de la conception VHDL par simulation

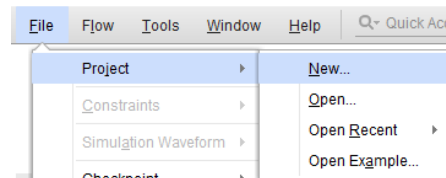
- Vérification de la conception du circuit avec un banc de test (testbench)

Partie 3 : mise en œuvre et génération du fichier binaire pour la cible FPGA

- Création d'un fichier de contraintes utilisateur (XDC)
- Synthèse, mise en œuvre et génération d'un fichier de programmation (bitstream)
- Programmation de la cible FPGA

Partie 1 : création d'un projet Vivado et d'une conception VHDL

- Pour démarrer l'IDE Vivado, sélectionnez **Start> Programs> Xilinx design tools> Vivado 2022.1** ou double-cliquez sur l'icône Vivado sur le bureau.



- Dans le navigateur de projet Vivado, cliquez sur **File> Project> New Project> Next**.

New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name: Premier_proj

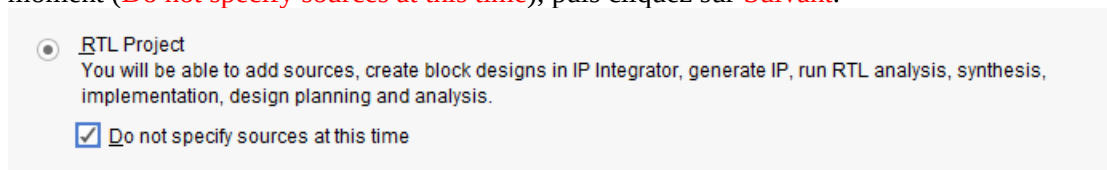
Project location: D:/Mahdi_Madani_Space/Teaching/FPGA_VHDL_M1/TP_MM/Projet_Prep_TPs

☒ Create project subdirectory

Project will be created at: D:/Mahdi_Madani_Space/Teaching/FPGA_VHDL_M1/TP_MM/Projet_Prep_TPs/Premier_proj

- Dans la fenêtre Assistant Nouveau Projet, saisissez TP1_Nom1_Nom2 comme nom du projet.
 - N'oubliez pas de choisir l'emplacement et de cocher la case **Create project subdirectory**.

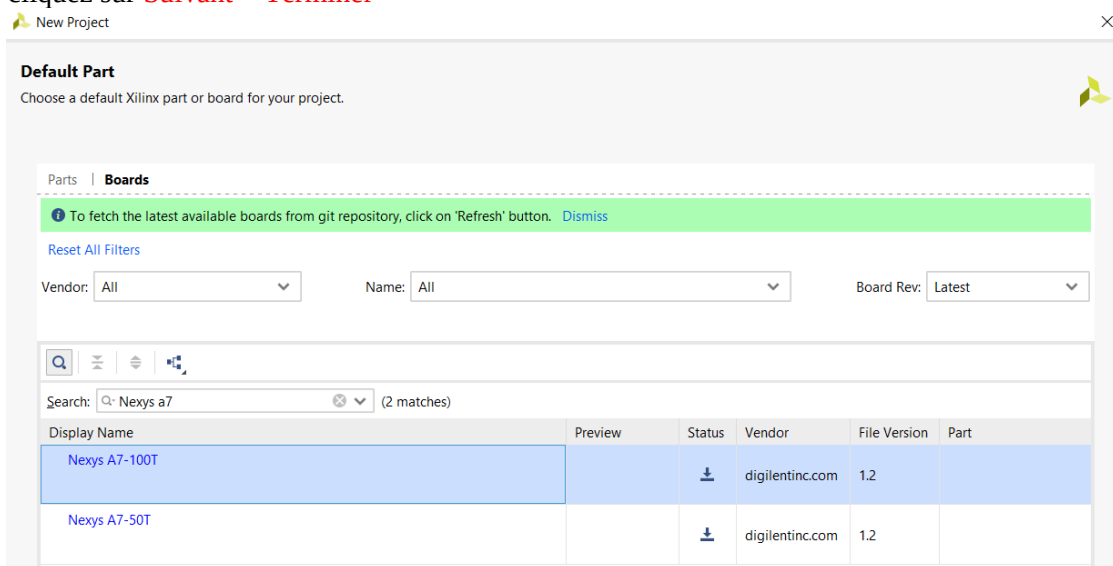
- Sélectionnez Projet RTL (**RTL Project**) > cochez la case Ne pas spécifier les sources pour le moment (**Do not specify sources at this time**), puis cliquez sur **Suivant**.



☐ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.

☒ **Do not specify sources at this time**

- Dans la nouvelle fenêtre, sélectionnez Cartes (**Boards**) et écrivez "**Nexys A**" dans la zone de recherche. Choisissez "**Nexys A7-100T**" (double-cliquez) et cliquez sur **Suivant > Terminer**. Si la carte n'est pas installée, cliquez sur la flèche pour la télécharger. Ensuite, double-cliquez et cliquez sur **Suivant > Terminer**



Default Part
Choose a default Xilinx part or board for your project.

Parts | **Boards**

To fetch the latest available boards from git repository, click on 'Refresh' button. [Dismiss](#)

[Reset All Filters](#)

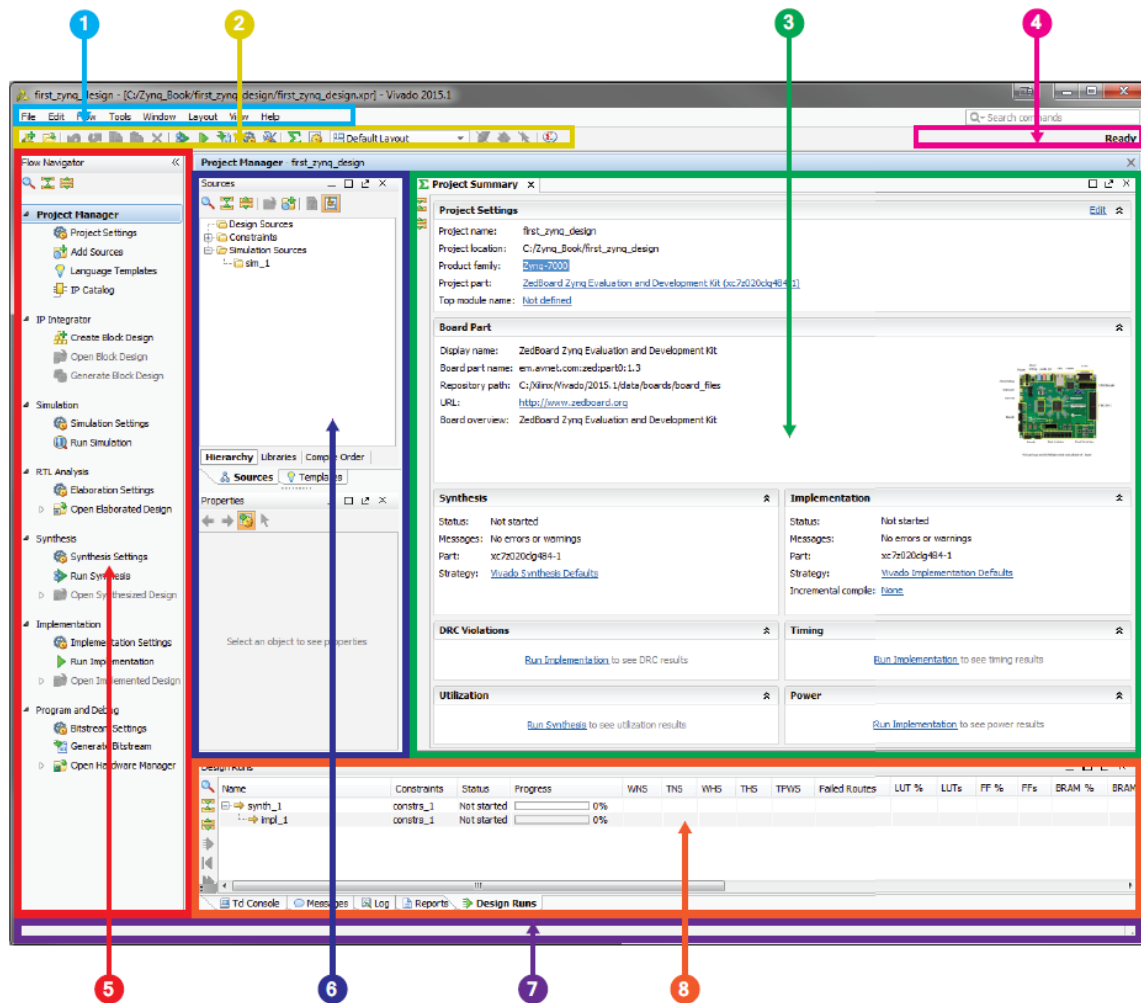
Vendor: All Name: All Board Rev: Latest

Search: Nexys a7 (2 matches)

Display Name	Preview	Status	Vendor	File Version	Part
Nexys A7-100T		↓	digilentinc.com	1.2	
Nexys A7-50T		↓	digilentinc.com	1.2	

Maintenant que nous avons créé votre premier projet dans l'IDE Vivado, vous pouvez passer à la création de votre première conception VHDL pour une cible FPGA.

Avant de le faire, je vous présente ci-dessous les différentes fenêtres de l'IDE Vivado.



En se référant aux numéros des étiquettes dans la Figure, voici les principaux modules de l'environnement de l'IDE Vivado :

- 1. Barre de menu** - La barre d'accès principal permet d'accéder aux commandes de l'IDE Vivado.
- 2. Barre d'outils principale** - La barre d'outils principale offre un accès facile aux commandes les plus couramment utilisées de l'IDE Vivado.
- 3. Espace de travail** - L'espace de travail offre une zone plus grande pour les panneaux qui nécessitent un espace d'écran plus important, tels que les panneaux graphiques, de schéma, de dispositif, de package et d'éditeur de texte.
- 4. Barre d'état de projet** - La barre d'état de projet du design actif actuellement.
- 5. Navigateur de flux** - Le navigateur de flux offre un accès facile aux outils et aux commandes nécessaires pour guider votre conception du début à la fin, en commençant par la section *Gestionnaire de projet* avec l'entrée de conception et se terminant par la génération de bitstream dans la section *Programmation et Débogage*. Les commandes d'exécution sont disponibles dans les sections Simulation, Synthèse et Implémentation.
- 6. Volet des fenêtres de données** - Le volet des fenêtres de données affiche par défaut des informations liées aux données de conception et aux sources, notamment :
 - Fenêtre des propriétés - Affiche des informations sur les objets logiques sélectionnés ou les ressources du dispositif.

- Fenêtre de la liste de connexions - Fournit une vue hiérarchique de la conception logique synthétisée ou élaborée.
- Fenêtre des sources - Affiche les sources IP, la hiérarchie, les bibliothèques et les vues de l'ordre de compilation.

7. Barre d'état - La barre d'état affiche diverses informations, notamment :

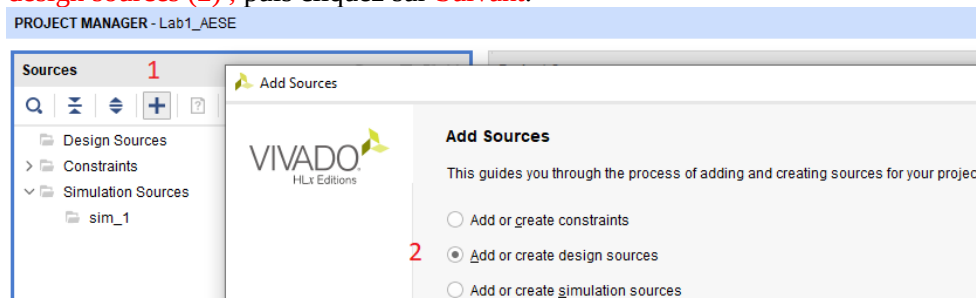
- Des informations détaillées concernant les commandes de la barre de menu et de la barre d'outils seront affichées dans le coin inférieur gauche de la barre d'état lorsque la commande est activée.
- Lorsque vous survolez un objet dans la fenêtre de schéma avec le pointeur de la souris, les détails de l'objet apparaissent dans la barre d'état.
- Lors de la création de contraintes et de placements dans les fenêtres de dispositif et de package, la validité et le type de contrainte seront affichés à gauche de la barre d'état. Les coordonnées du site et le type seront affichés à droite.
- La progression de la tâche en cours sera déplacée vers la droite de la barre d'état lorsque le bouton Arrière-plan est sélectionné.

8. Zone de fenêtre de résultats - La zone de fenêtre de résultats affiche l'état et les résultats des commandes dans un ensemble de fenêtres regroupées en bas de l'environnement de l'IDE Vivado. À mesure que les commandes progressent, des messages sont générés, des fichiers journaux et des rapports sont créés. Les informations connexes sont affichées ici. Les fenêtres par défaut comprennent :

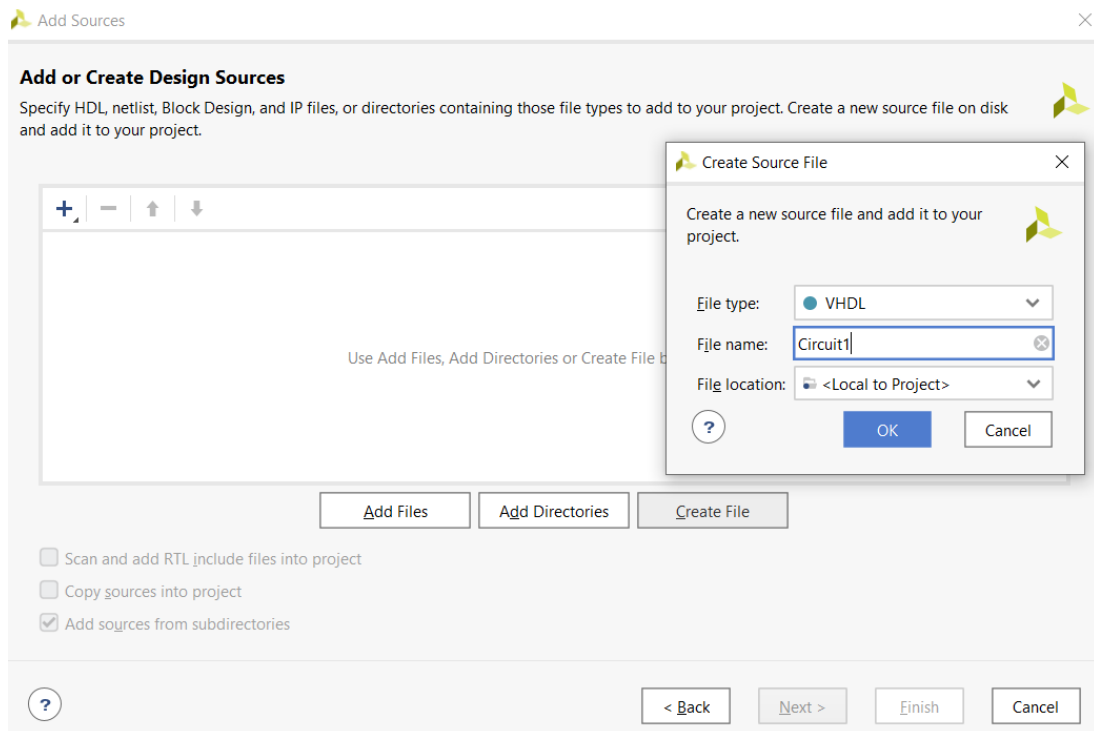
- **Messages** - Affiche tous les messages pour la conception active.
- **Console Tcl** - Les commandes Tcl peuvent être saisies ici, et un historique des commandes précédentes et des sorties est également disponible.
- **Rapports** - Un accès rapide aux rapports générés tout au long du flux de conception est fourni.
- **Journal** - Affiche les fichiers journaux générés par les processus de simulation, de synthèse et d'implémentation.
- **Exécutions de conception** - Gère les exécutions du projet en cours.

Avec l'introduction de l'environnement de l'IDE Vivado, vous pouvez maintenant passer à la création de la conception FPGA.

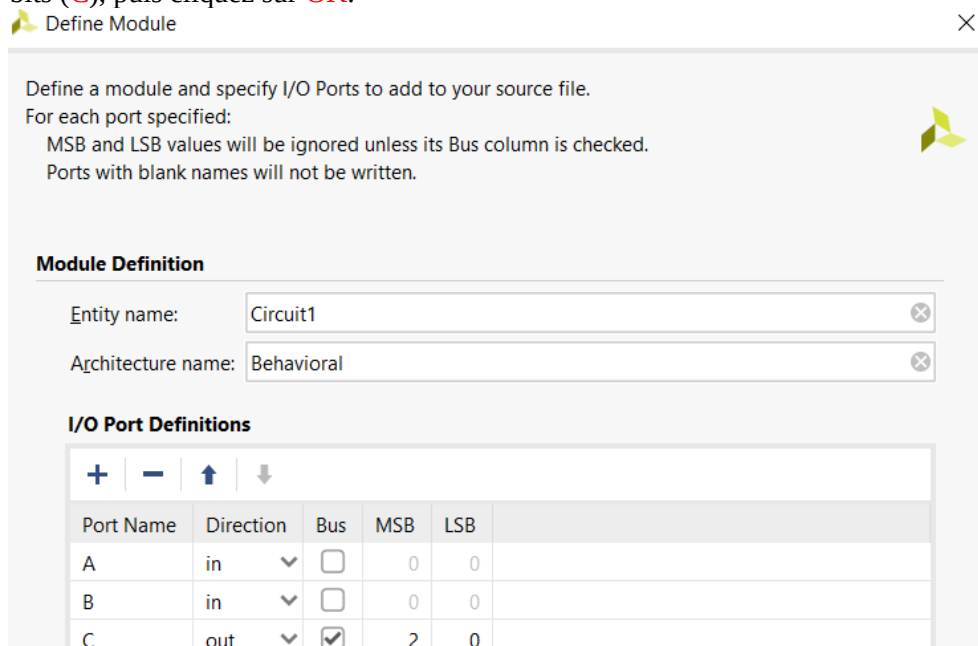
- Dans le Gestionnaire de projet Vivado, cliquez sur **Add sources (1)**, cochez la case **Add or crate design sources (2)**, puis cliquez sur **Suivant**.



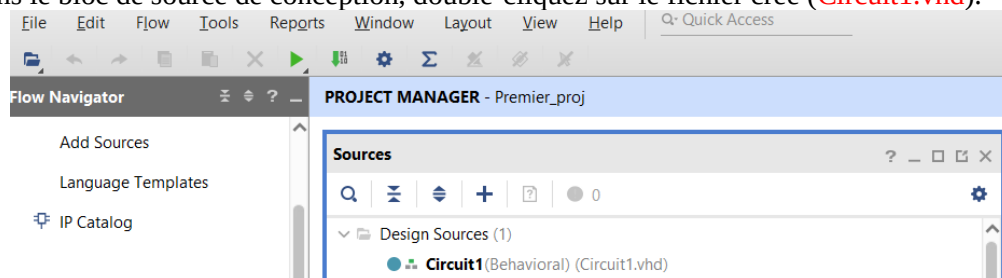
- Dans la fenêtre d'ajout de sources, cliquez sur **Create File (1)**, choisissez **VHDL** comme type (2), écrivez **Circuit1** comme nom de fichier (3), puis cliquez sur **OK (4)**, et enfin sur **Terminer**.



- Dans la fenêtre de définition du module, vous pouvez spécifier les ports d'entrée et de sortie de votre conception. Pour cette activité, ajoutez 2 entrées (A et B) et une sortie vectorielle sur 3 bits (C), puis cliquez sur **OK**.



- Dans le bloc de source de conception, double-cliquez sur le fichier créé (Circuit1.vhd).



- Complétez l'architecture VHDL comme suit:

```

1  -----
2  -- Company: Univ de Bourgogne
3  -- Engineer: Mahdi Madani
4  -- Create Date: 01/26/2024 11:38:15 AM
5  -- Design Name: Circuit1
6  -- Module Name: Circuit1 - Behavioral
7  -- Project Name: Premier_proj
8  -- Target Devices: Artix-7
9  -- Tool Versions: vivado 2022.1
10 -----
11 library IEEE;
12 use IEEE.STD_LOGIC_1164.ALL;
13
14 entity Circuit1 is
15     Port ( A : in STD_LOGIC;
16           B : in STD_LOGIC;
17           C : out STD_LOGIC_VECTOR (2 downto 0));
18 end Circuit1;
19
20 architecture Behavioral of Circuit1 is
21
22     begin
23         C(0) <= A or B;
24         C(1) <= A and B;
25         C(2) <= A xor B;
26
27     end Behavioral;
28 -----

```

Partie 2 : Vérification de la conception VHDL par simulation

Avant de mettre en œuvre une conception VHDL sur une cible FPGA, il est impératif de vérifier la fonctionnalité du système en simulant les combinaisons d'entrée/sortie possibles. La conception doit être vérifiée à l'aide d'un banc de test VHDL (**Test Bench, TB**). Il s'agit d'une partie de code utilisée pour vérifier la fonctionnalité d'un modèle HDL donné. Il instancie la conception à tester et génère des formes d'onde de stimulus pour la conception ou le modèle sous test. Il génère également des sorties de référence et les compare avec la sortie du modèle sous test. Ensuite, il fournit automatiquement une indication de réussite ou d'échec.

Avant de créer un banc d'essai, la conception VHDL du système sous test doit être préparée en suivant la Partie 1. Dans les étapes suivantes, vous créerez un banc de test pour le **Circuit1** conçu précédemment.

- Dans le Gestionnaire de projets Vivado, cliquez sur **Add sources**, cochez la case check **Add or create simulation sources**, puis cliquez sur **Suivant**.

 Add Sources

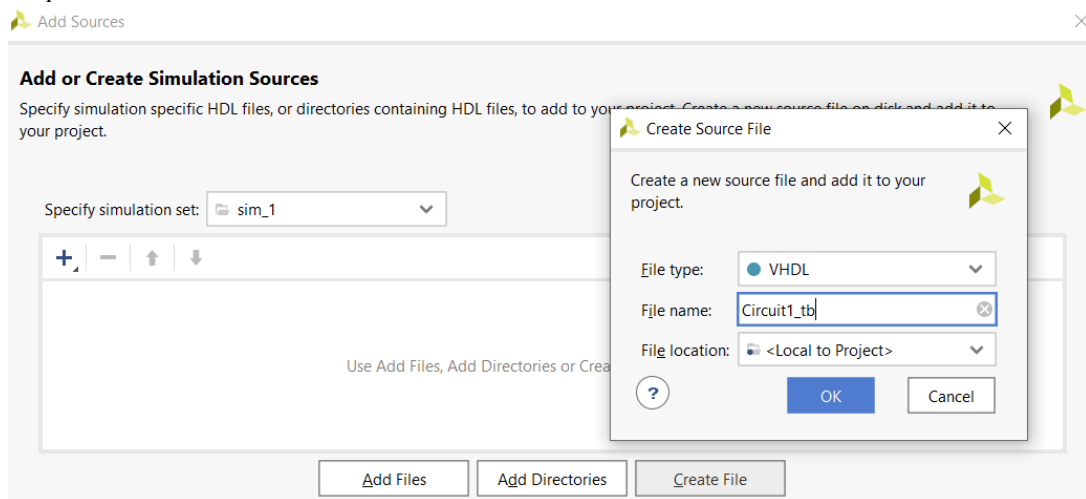


Add Sources

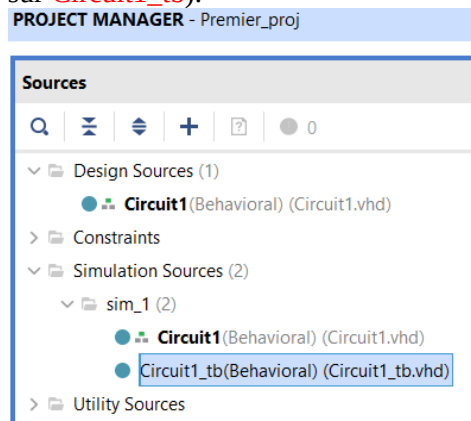
This guides you through the process of adding and creating sources for your project

- ☐ Add or create constraints
- ☐ Add or create design sources
- ☒ Add or create simulation sources

- Dans la fenêtre d'ajout de simulation, cliquez sur **Create File**, choisissez **VHDL** comme type, écrivez **Circuit1_tb** comme nom de fichier, puis cliquez sur **OK**, et enfin sur **Terminer**. Ensuite, cliquez sur **OK** et confirmez avec **Oui**.



- Dans la fenêtre des sources de simulation, vous trouverez votre fichier vide créé (double-cliquez sur **Circuit1_tb**).



- Pour simuler votre module, vous devez l'instancier avec le fichier testbench, comme suit.

```

3      -- Engineer: Mahdi Madani
4      -- Create Date: 01/26/2024 11:51:12 AM
5      -- Design Name: Circuit1
6      -- Module Name: Circuit1_tb - Behavioral
7
8      library IEEE;
9      use IEEE.STD_LOGIC_1164.ALL;
10
11     entity Circuit1_tb is
12     -- Port ( );
13     end Circuit1_tb;
14
15     architecture Behavioral of Circuit1_tb is
16     component Circuit1 is
17     Port ( A : in STD_LOGIC;
18           B : in STD_LOGIC;
19           C : out STD_LOGIC_VECTOR (2 downto 0));
20     end component;
21     signal A_test, B_test : STD_LOGIC := '0';
22     signal C_test : STD_LOGIC_VECTOR(2 downto 0) := "000";
23 begin
24     UUT : Circuit1 port map (A => A_test,
25                             B => B_test,
26                             C => C_test );
27
28     p_test : process
29     begin
30         A_test <= '0' ; B_test <= '0'; wait for 20 ns;
31         A_test <= '0' ; B_test <= '1'; wait for 20 ns;
32         A_test <= '1' ; B_test <= '0'; wait for 20 ns;
33         A_test <= '1' ; B_test <= '1'; wait ;
34     end process p_test;
35 end Behavioral;

```

- Assurez-vous que la hiérarchie de la simulation affiche l'architecture de votre testbench (**Circuit1_tb**) ainsi que l'instanciation du module que vous testez (**Circuit1**). La hiérarchie devrait ressembler à ceci :

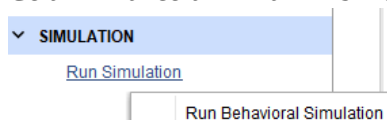
```

v Simulation Sources (1)
  v sim_1 (1)
    v  Circuit1_tb(Behavioral) (Circuit1_tb.vhd) (1)
       UUT : Circuit1(Behavioral) (Circuit1.vhd)

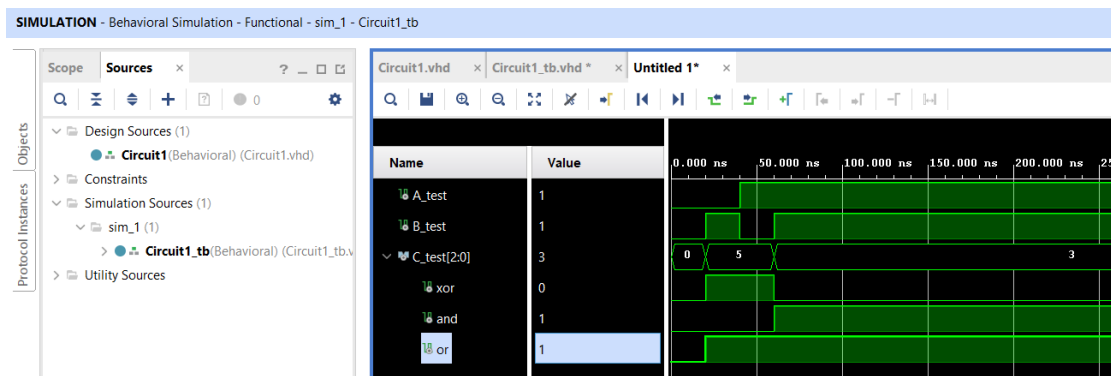
```

- Si la hiérarchie ne s'affiche pas correctement ou si le fichier de test (**Circuit1_tb**) n'est pas défini comme le module supérieur, Cliquez avec le bouton droit sur le fichier de test (**Circuit1_tb**) dans le Gestionnaire de projets. Ensuite, sélectionnez "**Set as top module**" (Définir comme module supérieur).
- Ensuite, suivez ces étapes pour exécuter la simulation comportementale de votre conception dans Vivado :

1. Ouvrez la fenêtre de conception dans Vivado.
2. Dans le volet de gauche, sous la section "**Simulation**," cliquez sur "**Run Simulation**".
3. Double-cliquez sur "**Run Behavioral Simulation**" (lancer la simulation comportementale). Cela lancera la simulation comportementale de votre conception.



- Assurez-vous d'avoir correctement configuré votre simulation en définissant le fichier de test comme module supérieur, comme indiqué précédemment. La simulation se déroulera en utilisant les signaux que vous avez définis dans votre banc de test pour vérifier le comportement de votre conception VHDL.
- Lorsque la simulation est lancée avec succès, la fenêtre de simulation s'ouvrira. Voici quelques actions que vous pouvez effectuer dans la fenêtre de simulation :



1. Pour afficher complètement la forme d'onde de toutes les entrées et sorties, cliquez sur l'icône "Zoom Fit" (Ajuster le zoom).
2. Dans la fenêtre de simulation, vous pouvez régler de nombreux paramètres, tels que la durée, le zoom (+, -), changer la couleur du signal, ajouter un signal interne, relancer la simulation, etc.
3. Pour ajuster la durée de la simulation, utilisez les commandes de défilement pour avancer ou reculer dans le temps.
4. Vous pouvez zoomer et dézoomer sur les signaux pour observer les détails de la forme d'onde.
5. Vous pouvez personnaliser la couleur des signaux pour une meilleure visualisation.
6. Il est possible d'ajouter des signaux internes pour la surveillance.
7. Si nécessaire, vous pouvez relancer la simulation pour effectuer d'autres tests ou apporter des modifications à votre conception.

Ces fonctionnalités vous permettront d'analyser en détail le comportement de votre conception VHDL lors de la simulation.

Partie 3 : Implémentation et génération du flux binaire pour programmer la cible FPGA

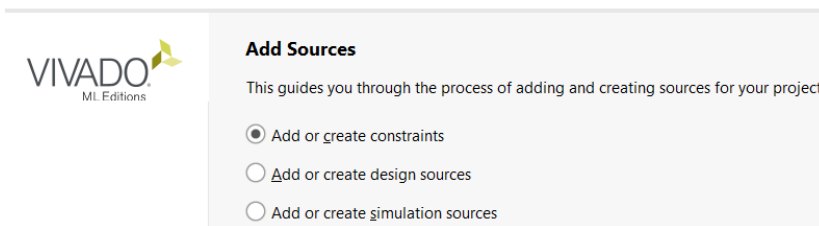
Dans cette partie, l'architecture conçue écrite en VHDL sera implémentée en ciblant le dispositif FPGA monté sur la carte **Nexys A7**. Les signaux 'A' et 'B' seront connectés aux switches **SW0** et **SW1**. Le vecteur de sortie 'C' sera connecté aux **LED** (LD2 à LD0) de la carte Nexys A7.

A. Creation du fichier de Constraints

Comme nous l'avons vu lors du cours, les FPGA possèdent des centaines de broches d'entrée/sortie. Par conséquent, il est important que les utilisateurs indiquent à l'outil de conception quelle broche du FPGA ils souhaitent utiliser pour leur conception. Ensuite, les outils de conception connecteront les entrées/sorties choisis aux interfaces de broches. Les outils Xilinx utilisent un fichier de contraintes (**fichier .xdc**) pour définir les contraintes utilisateur comme une broche physique pour circuiter les mappages de réseau. Ceci est parfois appelé fichier de contraintes d'implémentation. Ce fichier peut être modifié dans Vivado IDE à l'aide d'un éditeur de texte. Vous devez suivre les étapes suivantes pour créer le fichier de contraintes.

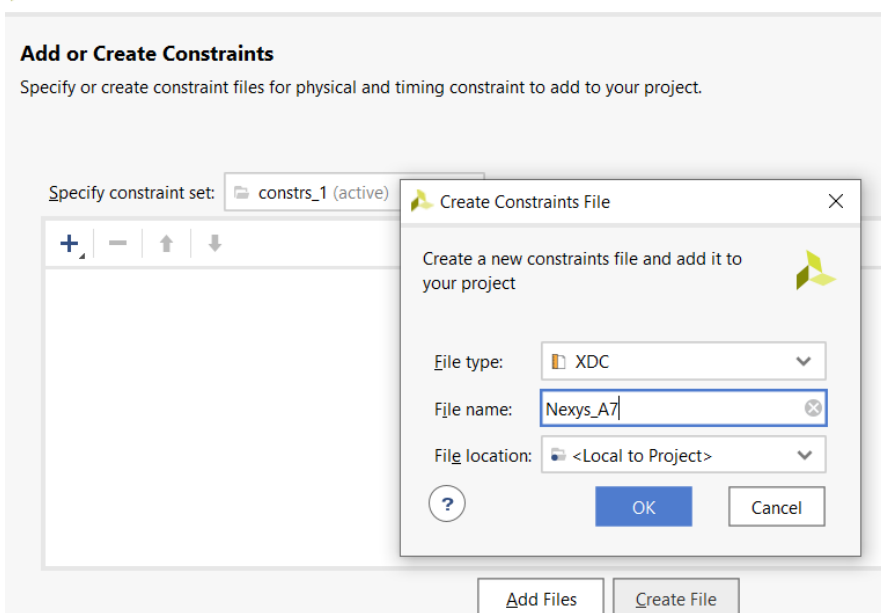
- Dans le gestionnaire de projet Vivado, cliquez sur **Add sources**, cochez le bouton **Add or create constraints**, puis cliquez sur **Suivant**.

 Add Sources

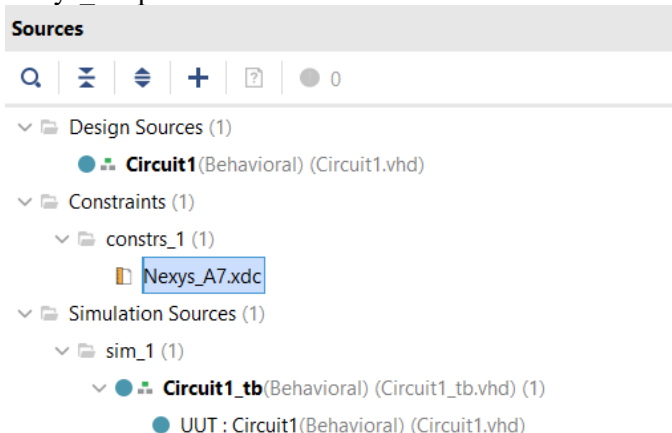


- Dans la fenêtre d'ajout de contraintes, cliquez sur **Create File**, choisissez **XDC** comme type et écrivez **Nexys_A7** dans le nom du fichier, cliquez sur **OK** et **Terminer**.

 Add Sources



- Dans la fenêtre des contraintes, vous retrouvez votre fichier vide créé, double-cliquez sur **Nexys_A7** pour l'ouvrir.



- Pour associer une broche physique à un signal de votre conception, tapez :
`set_property -dict { PACKAGE_PIN XX IOSTANDARD LVCMOS33 } [get_ports { netname }];`
 sur une ligne du fichier '.XDC'. Dans la déclaration, « **netname** » (guillemets inclus) est le nom signal à attacher à la PIN numéro XX (guillemets inclus).
 - Les numéros de broches de la carte Nexys A7 sont indiqués dans le manuel de référence.
 - Le fichier de contraintes complet de ce tutoriel est le suivant.

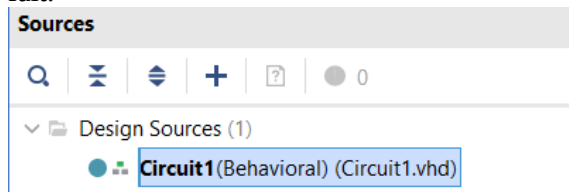
```

1  ##Switches
2  set_property -dict { PACKAGE_PIN J15    IOSTANDARD LVCMOS33 } [get_ports { A }];
3  set_property -dict { PACKAGE_PIN L16    IOSTANDARD LVCMOS33 } [get_ports { B }];
4
5  ## LEDs
6  set_property -dict { PACKAGE_PIN H17    IOSTANDARD LVCMOS33 } [get_ports { C[0] }];
7  set_property -dict { PACKAGE_PIN K15    IOSTANDARD LVCMOS33 } [get_ports { C[1] }];
8  set_property -dict { PACKAGE_PIN J13    IOSTANDARD LVCMOS33 } [get_ports { C[2] }];

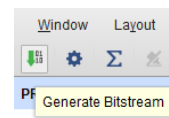
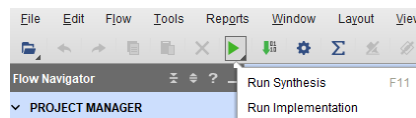
```

B. Génération du fichier binaire Bitstream

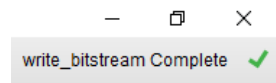
- Nous sommes maintenant prêts à créer un fichier de programmation (.bit) Pour cela, accédez à la fenêtre **Sources** et sélectionnez **Circuit1** comme module de niveau supérieur si ce n'est pas fait.



- Allez dans la fenêtre Processus où se trouvent deux processus particuliers consécutifs :
 1. Run Synthesis and Run implementation
 2. Generate Bitstream

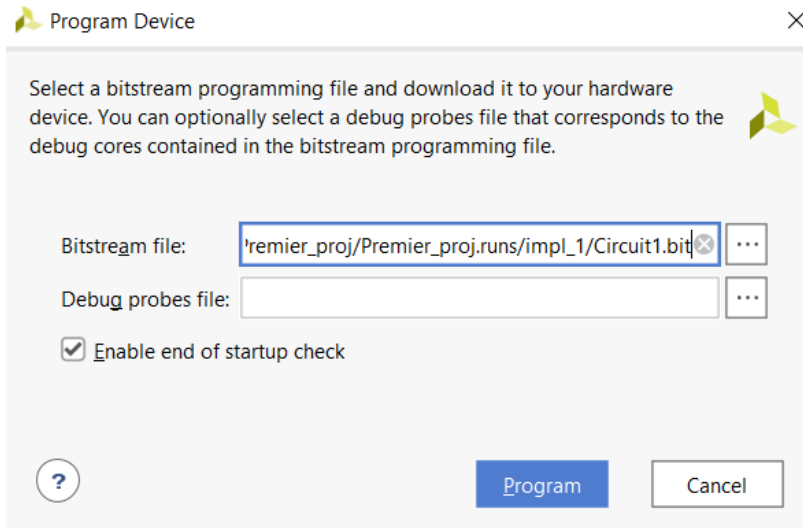
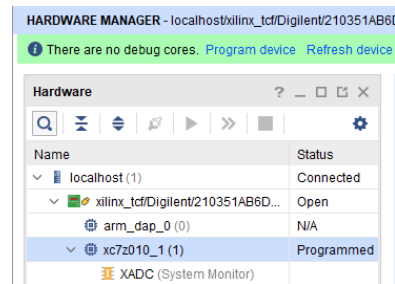
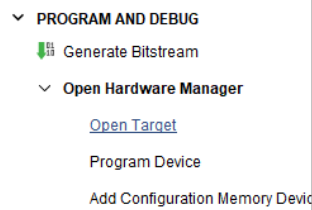


- Exécutez le processus de synthèse en double-cliquant sur le bouton **Run Synthesis**. Ce processus analyse le circuit que vous avez créé, en vérifiant les connexions, la syntaxe et la structure valide, pour vérifier que le circuit est valide et synthétisable.
- Si le processus de synthèse ne renvoie aucune erreur, vous pouvez continuer et exécuter l'implémentation. Ce processus utilise divers algorithmes pour cartographier le circuit numérique, puis crée des informations de lieu et de routage afin qu'il puisse être placé sur le FPGA physique.
- Vous pouvez double-cliquer directement sur le bouton **Generate Bitstream** et les 3 processus seront lancés séquentiellement et automatiquement.
- Si le fichier de programmation est généré correctement, vous aurez une indication verte en haut à droite de la fenêtre Vivado IDE.



C. Programmer le FPGA avec le mode JTAG

- Pour programmer le FPGA en mode JTAG avec le fichier **.BIT** généré à l'étape précédente. Connectez le câble USB entre votre poste de travail et la prise USB de la carte Nexys A7 et allumez la carte à l'aide de l'interrupteur marche/arrêt.
- Procédez comme suit pour programmer le FPGA :
- Cliquer sur **Open Target** > **Auto Connect** > **Program device** > vérifier que le chemin de votre fichier bitstream est correct > cliquer sur **Program**.



Felicitacion! Vous avez maintenant programmé avec succès votre premier composant FPGA sous Vivado ! Il vous rest qu'à vérifier son bon fonctionnement.

Exercice :

Supposons qu'une pièce comporte deux ampoules (LED0 et LED1) et trois interrupteurs (SW0, SW1 et SW2). La lumière à l'intérieur de cette pièce est contrôlée comme suit :

- La LED0 s'allume lorsque l'un des trois interrupteurs est allumé, sinon elle reste éteinte.
- La LED1 est allumée lorsqu'au moins deux interrupteurs sont allumés, sinon elle doit être éteinte.

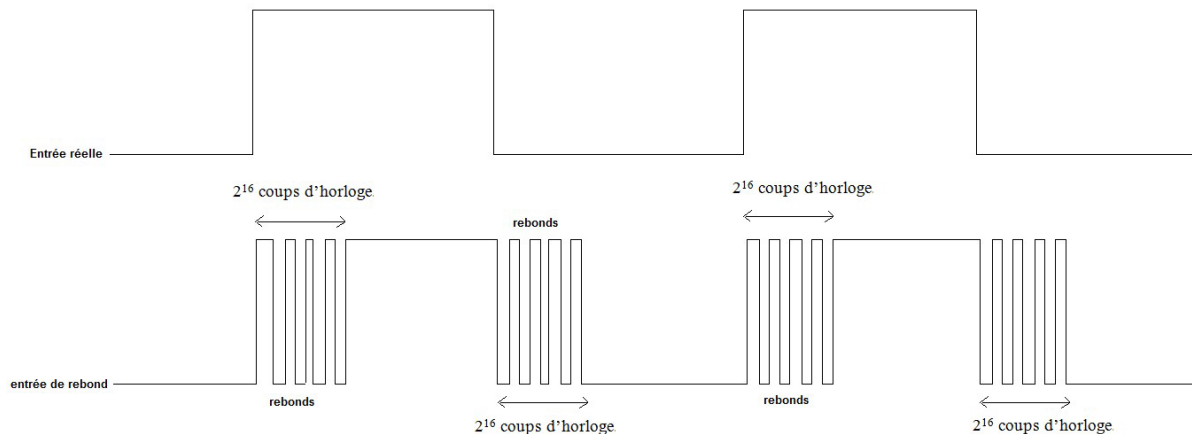
Concevez un circuit numérique permettant d'assurer le control de la lumière au sein de cette pièce.

Créez un nouveau projet et suivez les mêmes étapes que le tutoriel :

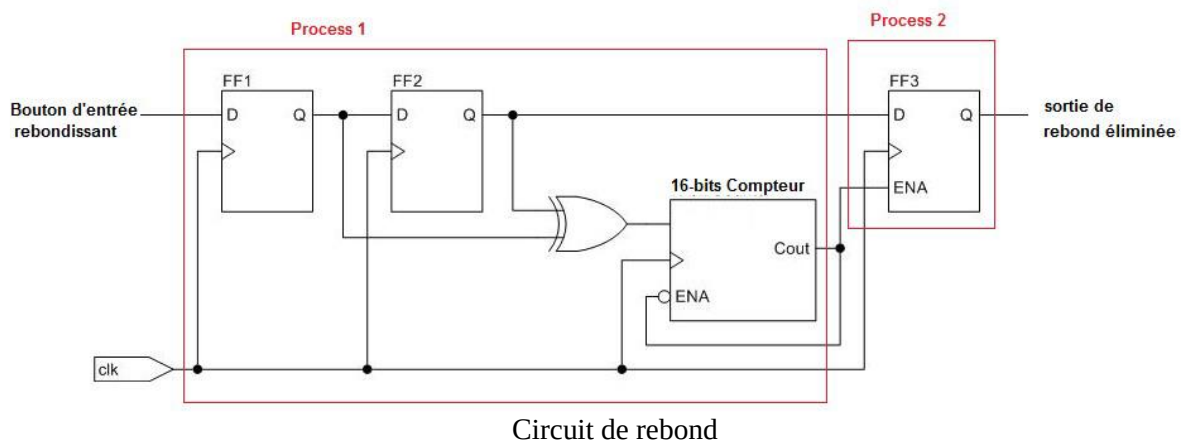
- Ajoutez un fichier source et complétez-le avec l'entité et l'architecture du circuit.
- Ajouter un fichier de simulation et vérifier son bon fonctionnement.
- Implémenter-le sur la carte et vérifier le comportement réel de votre circuit.

TP2 : Filtre anti-rebond sur FPGA (1 séance)

Il existe un phénomène de rebond lorsqu'on utilise des boutons mécaniques. Au moment de la connexion ou de la déconnexion, les boutons rebondissent et faussent le résultat. Il est nécessaire d'éliminer les rebonds possibles pour prendre en compte correctement l'état de ces boutons.



Pour cela, vous allez créer un design en VHDL possédant deux niveaux de hiérarchie. Vous allez tout d'abord créer un composant qui aura pour rôle d'éliminer les rebonds sur 5 boutons différents. Ce composant recevra en entrée les états des 5 boutons et l'horloge globale à 100 MHz, et fournira en sortie les états des 5 boutons après filtrage. On considérera un état stable pour un bouton si celui-ci ne change pas dans 2^{16} coups d'horloge.



Pour ceci, vous créez donc, pour chaque bouton, deux process.

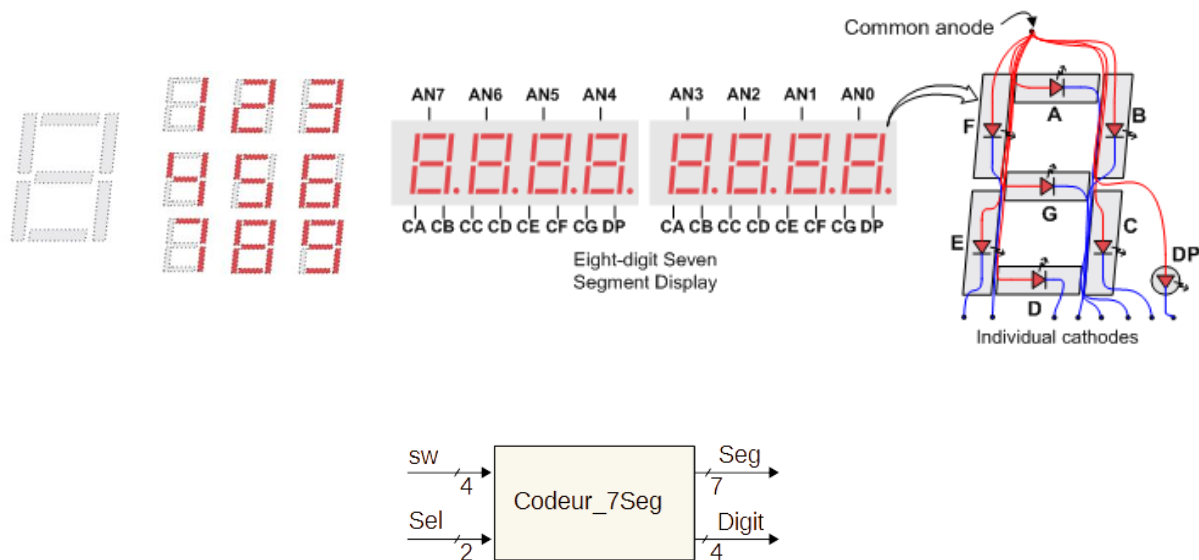
1. Le premier process consiste à incrémenter un compteur lorsque la valeur courante du bouton est la même que celle du bouton mémorisé (sinon le compteur est remis à 0).
2. Le second process changera la valeur du bouton mémorisé si le compteur précédent est arrivé à sa valeur maximum.

Validez le fonctionnement sur la carte en allumant et éteignant les LED en fonction de l'appui sur les boutons.

TP3: Contrôle d'un afficheur 7 segments par un FPGA (1 séance)

Dans ce TP, nous expérimentons les techniques classiques de synthèse combinatoire. Dans la première partie, nous réalisons un transcodeur du code binaire naturel vers un afficheur 7 segments. Dans la deuxième partie, nous implémentons ce circuit sur un FPGA.

La carte Nexys A7 dispose d'un affichage à sept segments à quatre chiffres. Et les sept LEDs de segment sont configurées comme actives à l'état bas, ce qui signifie qu'un segment de LED est allumé si le signal de commande correspondant est '0'.



Créez un design en VHDL permettant d'afficher la valeur binaire formée par les 4 premiers commutateurs de la carte sur l'un des 8 afficheurs 7 segments disponibles.

1. Vous pouvez commencer par remplir la table de vérité correspondante.

Entrée				Sortie						
Sw3	Sw2	Sw1	Sw0	a	b	c	d	e	f	g
...	...	...	...	...	...	...	...	...	...	...

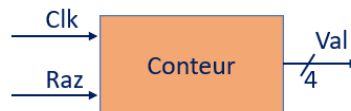
2. Ensuite, écrivez et testez le code VHDL correspondant à ce circuit.
3. Notez que si vous souhaitez sélectionner le premier afficheur, vous devez fixer la valeur de Digit à "1110", et ainsi de suite pour les autres.

TP3: Systèmes séquentiels (2 séances)

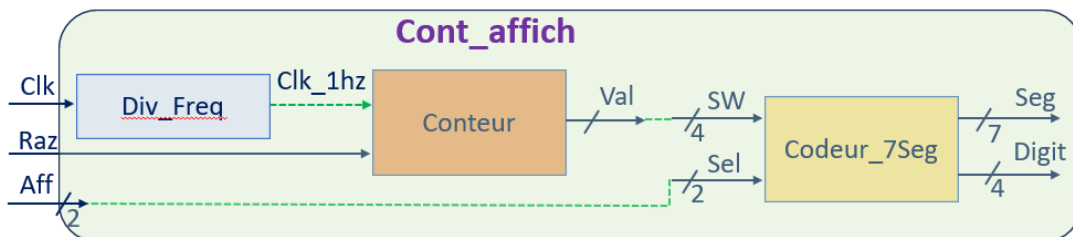
Durons deux séances, vous allez concevoir différents systèmes séquentiels commençant par des systèmes de base. Ensuite vous finissez avec des systèmes plus avancés.

Exercice 1 :

1. Concevez un compteur qui vous permet d'afficher les chiffres de 0 à 9 sur l'un des afficheurs 7 segments de la carte Nexys A7. L'état du compteur s'incrémentera toutes les secondes (il faut concevoir un diviseur en VHDL et le simuler) et il sera réinitialisé lorsque le bouton-poussoir central de la carte Nexys-A7 est enfoncé.
 - a. Pour cela, vous devez créer un module VHDL pour le conteur asynchrone modulo 10. Et simulez votre module avec un banc de test pour s'assurer de son bon fonctionnement.



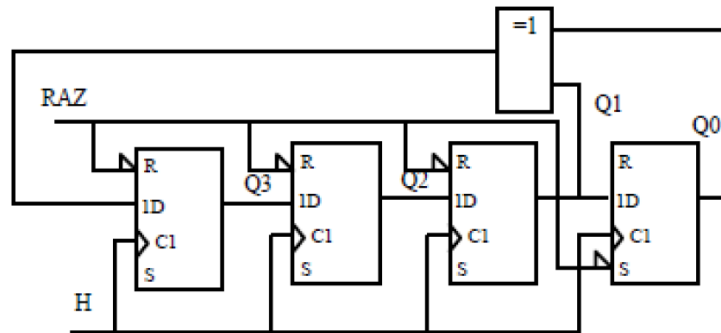
- b. Ensuite, connectez le module conteur avec le module codeur 7 segments du TP2 sous un module principal (Cont_affich) qui assure leur connexion. Simulez par un nouveau banc de test. Synthétisez, implémentez, générez le fichier binaire et programmez le FPGA sur la carte Nexys A7.



2. Pour améliorer votre compteur, ajoutez une fonction de contrôle par les 5 boutons poussoirs. Effectuez les modifications nécessaires pour obtenir un compteur incrémenté ou décrémenté en fonction des boutons haut et bas de la carte. Vous afficherez également le résultat du compteur en passant d'un affichage à l'autre à l'aide des boutons poussoirs gauche et droit. Si vous appuyez sur le bouton gauche, le résultat doit s'afficher en se décalant d'un pas vers la gauche. Si vous appuyez sur le bouton de droite, votre résultat doit s'afficher sur l'écran suivant à droite.
3. Pour finir, étendez la conception ci-dessus pour utiliser l'affichage à 4 chiffres simultanément pour afficher des nombres allant de 0 à 9999.

Exercice 2 :

1. Observez bien la figure ci-dessous (en particulier le signal de réinitialisation RAZ), puis implémentez le circuit en VHDL.
2. Simulez votre circuit avec le banc de test correspondant.
3. Déterminez la séquence générée en binaire et en décimal (Q3Q2Q1Q0). Notez que RAZ est d'abord réglé sur un niveau bas puis sur un niveau haut en permanence.
4. Quelle est la fonction de ce circuit ?
5. Initialisez le circuit à 0000, puis refaire les simulations. Que se passe-t-il ?



Exercice 3 :

Dans cette activité, vous allez créer une minuterie qui peut estimer une durée de 4 minutes à 0 seconde.

- Concevez une minuterie qui vous permet de décompter une durée définie par les huit commutateurs de la carte Nexys_A7. Les six premiers représentent les secondes et les deux autres représentent les minutes. Le résultat est affiché sur trois des afficheurs 7 segments existants. La minuterie commence à compter (décrémenter chaque seconde) lorsque nous appuyons sur le bouton-poussoir droit. Il s'arrête lorsque nous appuyons sur le bouton poussoir gauche. Et il se réinitialise lorsque nous appuyons sur le bouton-poussoir central. Nous utilisons également quatre LED comme signal lumineux pour indiquer à l'utilisateur la progression du temps. Au début, les 4 LED sont allumées. Ensuite, lorsque 25% de la durée choisie s'est écoulée, la LED de gauche s'éteint, et ainsi de suite, jusqu'à ce que toutes les LED s'éteignent.
 - Concevez ce minuteur, puis implémentez-le en VHDL.
 - Simulez avec un banc de test pour tester toutes les fonctionnalités évoquées.
 - Générez le bitstream et programmez le FPGA pour vérifier que votre circuit répond aux attentes.

TP5: Machine à états finis (1 séance)

Cette séance aborde la description d'une machine à états en langage VHDL. Elle doit vous permettre de bien comprendre les différentes étapes de la synthèse d'un système séquentiel décrit à l'aide d'un formalisme de type graphe d'états.

Cahier des Charges du système à réaliser

Nous souhaitons réaliser le système de commande d'une machine qui vend des films pour 5\$. Le contexte est le suivant :

- L'utilisateur a le droit de mettre des pièces de 1\$ ou de 2\$ et dès que le montant arrive à 5\$ ou plus, il y a un film qui sort.
- Il est possible que l'utilisateur mette 6\$, et dans ce cas, la machine donne un film et remet aussi la monnaie.
- Notez que si la personne mettait de l'argent pendant que le film sort, l'argent sera perdu.
- Nous voulons réaliser ce système à l'aide d'une machine de Moore (les sorties ne dépendent que de l'état).

Pour vous faciliter la conception, nous vous donnons ci-dessous une partie du code VHDL à améliorer.

- Avant de passer au code VHDL, déterminez le graphe d'états de transitions de ce système.
- Vous remarquerez que l'entité contient 3 entrées *un_dollar*, *deux_dollars*, *clk* et deux sorties *film* et *monnaie*.

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
ENTITY machine_etats_top IS
PORT (
    clk          : IN STD_LOGIC;
    un_dollar    : IN STD_LOGIC;
    deux_dollars : IN STD_LOGIC;
    film         : OUT STD_LOGIC;
    monnaie      : OUT STD_LOGIC
);
END machine_etats_top;

ARCHITECTURE Behavioral OF machine_etats_top IS

TYPE etats IS (etat_0, etat_1, etat_2, etat_3, etat_4, etat_5, etat_6);
SIGNAL etat_present : etats;
SIGNAL etat_prochain : etats;

BEGIN

Pro_1:PROCESS (etat_present, un_dollar, deux_dollars)
BEGIN
    CASE etat_present IS
        WHEN etat_0 =>
            film <= '0';
            monnaie <= '0';
            IF un_dollar = '1' THEN
                etat_prochain <= etat_1;
            ELSIF deux_dollars = '1' THEN
                etat_prochain <= etat_2;
            ELSE
                etat_prochain <= etat_0;
            END IF;
        WHEN etat_1 =>
            ....

        WHEN etat_6 =>
            film <= '1';
            monnaie <= '1';
            etat_prochain <= etat_0;
        END CASE;
    END PROCESS;

Pro_2:PROCESS (clk)
BEGIN
    IF clk'EVENT AND clk = '1' THEN
        etat_present <= etat_prochain;
    END IF;
END PROCESS;

END Behavioral;

```

Pour tester le programme sur la carte Nexys-A7, on peut connecter les entrées *un_dollar* et *deux_dollars* aux boutons poussoirs et les sorties *film* et *monnaie* aux LEDs.

Exemple : si l'utilisateur appuie trois fois sur les boutons correspondent à *deux_dollars*, la machine sera à l'*état_6* et le film et la monnaie seront activés. Si l'utilisateur, appuie sur le bouton correspond à *un_dollar* cinq fois, la machine sera à l'*état_5* et seul le film sera activé.

Cependant, on doit utiliser une horloge de 1 Hz pour visualiser le résultat sur la carte. Par conséquent, vous devez :

- Ajoutez un nouveau processus dans le code ci-dessus pour générer une horloge de 1 Hz et l'utiliser dans le processus *pro_2*.
- Ajoutez une sortie *etat* à l'entité et l'affichez la sur l'un des afficheurs 7 segments.
- Référez-vous à l'annexe pour la simulation en tastbench.

Annexe : simulation par un banc de test

Comme évoqué durs le cours, pour simuler un module VHDL, on utilise un banc de test. Dans cet exemple, le banc de test ne contient deux processus et aucune assignation de signaux de test.

- Le premier processus consiste à créer la période d'horloge.
- Le deuxième processus consiste à attribuer la valeur des autres signaux à différentes périodes de l'horloge.

Vous allez ajouter des assignations le processus marqué par un rectangle rouge ci-dessus :

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY machine_etats_top_tb IS
END machine_etats_top_tb;

ARCHITECTURE behavior OF machine_etats_top_tb IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT machine_etats_top
    PORT (
        clk : IN std_logic;
        un_dollar : IN std_logic;
        deux_dollars : IN std_logic;
        film : OUT std_logic;
        monnaie : OUT std_logic;
        etat : OUT std_logic_vector(2 downto 0)
    );
    END COMPONENT;

    --Inputs
    signal clk : std_logic := '0';
    signal un_dollar : std_logic := '0';
    signal deux_dollars : std_logic := '0';

    --Outputs
    signal film : std_logic;
    signal monnaie : std_logic;

    -- Clock period definitions
    constant clk_period : time := 10 ns;

BEGIN

    -- Instantiate the Unit Under Test (UUT)
    uut: machine_etats_top PORT MAP (
        clk => clk,
        un_dollar => un_dollar,
        deux_dollars => deux_dollars,
        film => film,
        monnaie => monnaie
    );

    -- Clock process definitions
    clk_process : process
    begin
        clk <= '0';
        wait for clk_period/2;
        clk <= '1';
        wait for clk_period/2;
    end process;
```

Un exemple de process test :

```
-- Stimulus process
stim_proc: process
begin
    -- hold reset state for 100 ns.
    wait for 50 ns;

    un_dollar <= '0';
    deux_dollars <= '1';
    wait for clk_period*5;

    un_dollar <= '0';
    deux_dollars <= '0';
    wait for clk_period*3;

    un_dollar <= '1';
    deux_dollars <= '0';
    wait for clk_period*5;

    un_dollar <= '0';
    deux_dollars <= '0';
    wait for clk_period*5;

    un_dollar <= '1';
    deux_dollars <= '1';
    wait for clk_period*5;

    un_dollar <= '0';
    deux_dollars <= '0';
    wait for clk_period*3;

    un_dollar <= '1';
    deux_dollars <= '0';
    wait for clk_period*3;
end process;
```

```
-- Stimulus process
stim_proc: process
begin
    -- hold reset state for 100 ns.
    wait for 100 ns;

    wait for clk_period*10;

    -- insert stimulus here
    wait;
end process;
```

END;